

Pneumonia Classification using Chest XRay Images

By: Hritesh Rajanagan, Steven Huynh, Ethan Kittell

The below code is the complete model pipeline for the proposed solution model (DeiT vision transformer). The following sections outline data visualization, data preprocessing, model definitions, training loops, evaluation loops, and finally model analysis. Instructions for downloading the required dataset can be found in the readme file.

Required packages

If this is the first run and the "final_train" directory is not yet created follow these steps:

- download the dataset from kaggle at:
<https://www.kaggle.com/datasets/kostasdiamantaras/chest-xrays-bacterial-viral-pneumonia-normal>
- create a new virtual environment
- install all the below required packages
- uncomment all code blocks to create the "final_train" directory
- run from start to end

If the "final_train" directory already exists

- ensure the virtual environment contains the required packages
- run the code from start to end.

```
In [2]: import torch
from torch import nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import numpy as np
import os

from transformers import DeiTForImageClassification
from transformers import DeiTConfig, DeiTModel
from torchvision import datasets, transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader, random_split
from collections import Counter
import torch.nn as nn
import torch.optim as opt
import torch.optim.lr_scheduler as lr_scheduler
import timm
```

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support
import scipy.ndimage
from scipy import misc
from glob import glob
from scipy import stats
from sklearn.preprocessing import LabelEncoder, StandardScaler
import skimage
import imageio
import seaborn as sns
from PIL import Image
import glob
import matplotlib.pyplot as plt
import matplotlib
import cv2

device = torch.device('cuda:3' if torch.cuda.is_available() else 'cpu')
device

```

```

/home/public/shuynh/HW_7/newenv/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
/tmp/ipykernel_3198622/1299914090.py:25: DeprecationWarning: scipy.misc is deprecated and will be removed in 2.0.0
  from scipy import misc

```

Out[2]: device(type='cuda', index=3)

Data Visualization

Before data preprocessing, visualize the dataset for key insights like class imbalance, input shape, noise, and other key features that would impact the quality of the later analysis.

1. visualize the provided metadata. This allows the user to see how images and class labels are linked.

```

In [ ]: #Getting a summary of the number of images and the different class IDs
df = pd.read_csv('archive/labels_train.csv')
df.shape
df

```

Out[]:

	file_name	class_id
0	img_4358977458434011046.jpg	1
1	img_5224016757187192130.jpg	2
2	img_3065202206106254707.jpg	2
3	img_6304894865561547174.jpg	1
4	img_3371338542810939877.jpg	2
...	...	...
4667	img_6661311872293090412.jpg	2
4668	img_3844568579349757418.jpg	1
4669	img_9145812369383814369.jpg	1
4670	img_1311393330250392648.jpg	1
4671	img_3089041144907674826.jpg	1

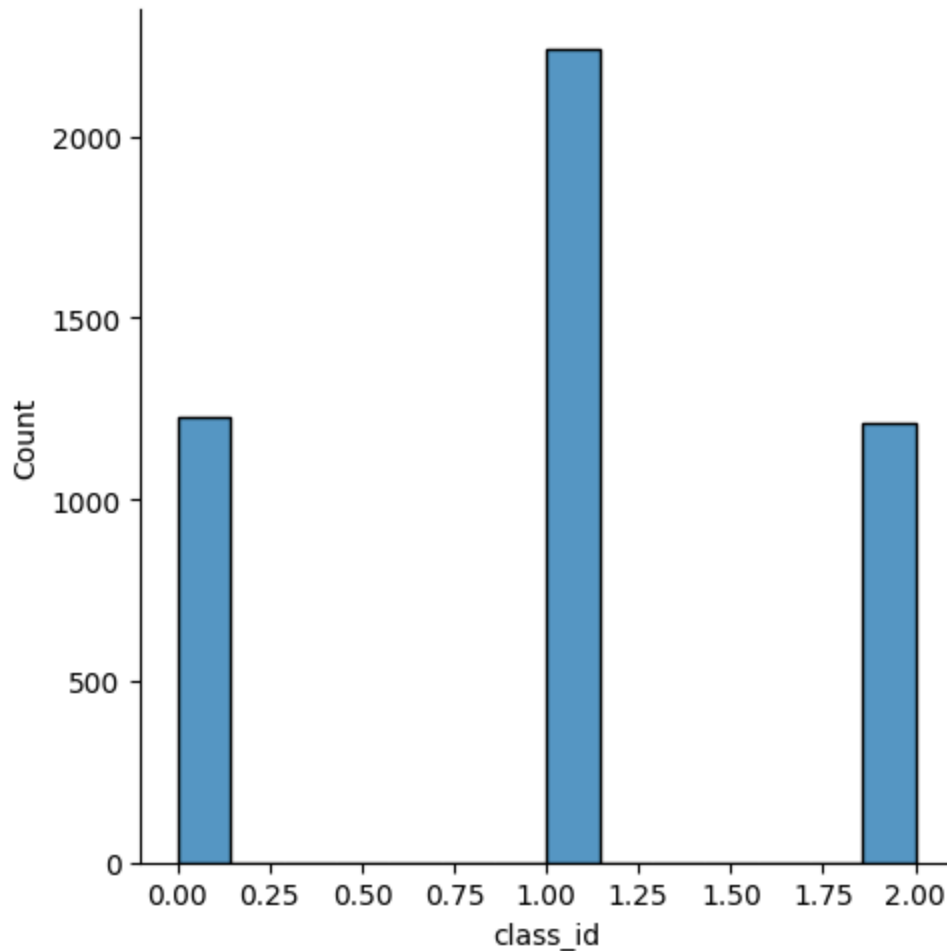
4672 rows × 2 columns

```
In [21]: # Need to create folders for each X-ray type to store the images
types = df['class_id'].unique()
print(types)
```

[1 2 0]

2. visualize the distribution of class labels/images across the dataset to inform handling class imbalance. The visualization reveals that the classes are imbalanced at a roughly 2:1:1 ratio of bacterial/no-pneumonia/viral samples.

```
In [22]: # visualization displot of the different classes and their image counts
sns.displot(df['class_id'])
plt.show()
```



3. to address the class imbalance seen above, class weights are generated to help guide the loss function later defined before the training loop. By analyzing the histogram, appropriate weights help penalize frequent classes more than others.

In [132...

```
# sizes of each class
classes = df['class_id'].unique()
total_images = df.shape[0]
class_weights = {}
class_frequencies = []

for c in classes:
    class_weights[c] = df['class_id'].value_counts().get(c, 0)
    class_frequencies.append(class_weights[c]/total_images)
    class_weights[c] = class_weights[c]/total_images

class_median = np.median(class_frequencies)

for c in class_weights:
    class_weights[c] = class_median/class_weights[c]
    print(f"{c} : {class_weights[c]}")

sorted_classes = sorted(class_weights.keys())
weights_list = [class_weights[c] for c in sorted_classes]
weights_tensor = torch.FloatTensor(weights_list)
```

```
print(sorted_classes)
```

```
1 : 0.5482573726541554
2 : 1.0165700082850042
0 : 1.0
[np.int64(0), np.int64(1), np.int64(2)]
```

4. To appropriately visualize data, rename the images and move them to the appropriate class folders. By doing this step, the dataset remains organized and easy for users to read. This format also improves programming experience as all images are properly labeled and easily accessible.

```
In [ ]: import os
import shutil

disease_types = df['class_id'].unique()
folders = {}
NP_count = 0
BP_count = 0
VP_count = 0

# ---- UNCOMMENT THE BELOW CODE ON A FIRST RUN ----

# for type in disease_types:
#     if type == 0:
#         folder_name = 'no_pneumonia'
#     elif type == 1:
#         folder_name = 'bacterial_pneumonia'
#     else:
#         folder_name = 'viral_pneumonia'

#     folders[type] = folder_name

#     if not os.path.exists("final_train/" + folder_name):
#         os.makedirs("final_train/" + folder_name)
#     else:
#         print("already exists")

# each image from the chest X-ray dataset
# for filename in os.listdir('archive/train_images'):
#     image = df.loc[df['file_name'] == filename]
#     img_class = image['class_id'].values[0]
#     src = 'archive/train_images/' + filename

#     if img_class == 0:
#         folder = "no_pneumonia"
#         file_count = NP_count
#         NP_count += 1
#     elif img_class == 1:
#         folder = "bacterial_pneumonia"
#         file_count = BP_count
#         BP_count += 1
#     else:
```

```
#         folder = "viral_pneumonia"
#         file_count = VP_count
#         VP_count += 1

#     dest = 'final_train/' + folder + '/' + filename
#     shutil.copy(src, dest)

# print("Done!")

# for folder in folders.values():
#     num_images = len(os.listdir('final_train/' + folder))
#     print(f"{folder}: {num_images} total images")
```

5. rename the files for ease of identification

```
In [ ]: NP_count = 0
BP_count = 0
VP_count = 0

# ---- UNCOMMENT THE BELOW CODE ON A FIRST RUN ----

# for folder in folders.values():
#     for filename in os.listdir('final_train/' + folder):

#         if folder == "no_pneumonia":
#             file_count = NP_count
#             NP_count += 1
#         elif folder == "bacterial_pneumonia":
#             file_count = BP_count
#             BP_count += 1
#         else:
#             file_count = VP_count
#             VP_count += 1

#         new_name = 'final_train/' + folder + '/' + folder + '_' + str(file_count)
#         old_name = 'final_train/' + folder + '/' + filename
#         os.rename(old_name, new_name)
```

6. Display a batch of images for each class to visualize how the Chest X Rays look. The visualization informs the user that viral and normal samples look similar to each other explaining future difficulties in classification

```
In [ ]: label = ['0', '1', '2']
label_images = []
classes = ['no_pneumonia', 'bacterial_pneumonia', 'viral_pneumonia']

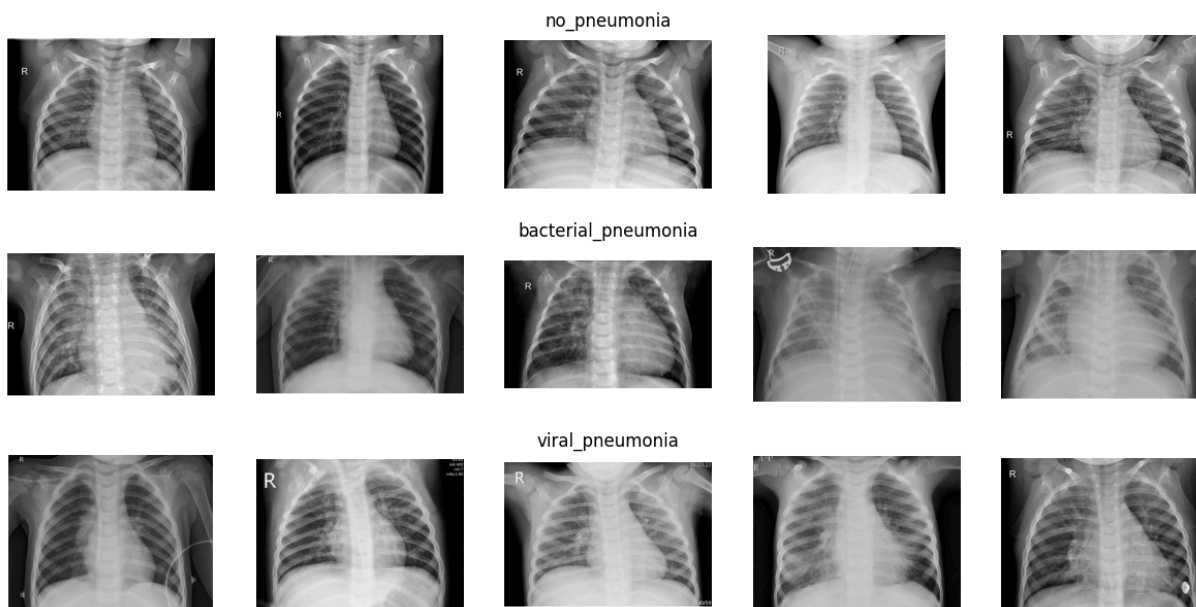
for index, label in enumerate(classes):
    plt.figure(figsize=(15,2))
    plt.title(classes[index])
    plt.axis('off')
```

```

for i in range(5):
    img = plt.imread('final_train/' + label+ '/' + os.listdir('final_train/' +
    plt.subplot(1, 5, i +1)
    plt.axis('off')
    plt.imshow(img, cmap="gray")

plt.show()

```



- visualize individual images of each class to determine levels of noise. Noise is present and initially guided transformations in the data preprocessing phase. However, in conducting further evaluations, the noise did not greatly effect the quality of the image. Therefore, noise was not considered during preprocessing. The noise also adds extra variance, which helps the model during training.

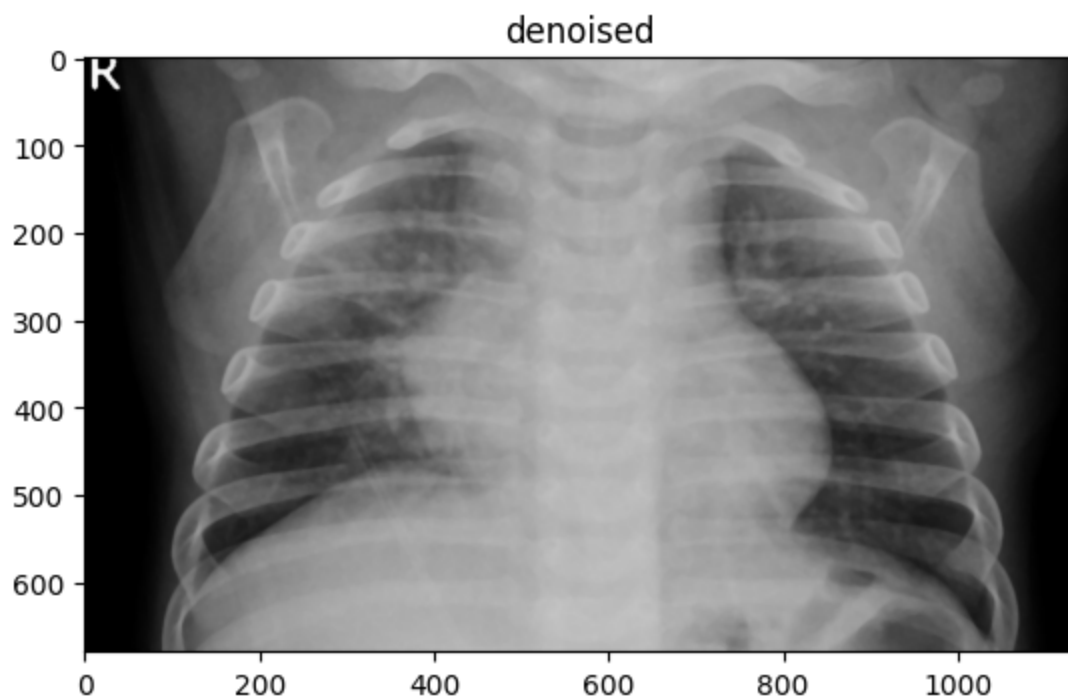
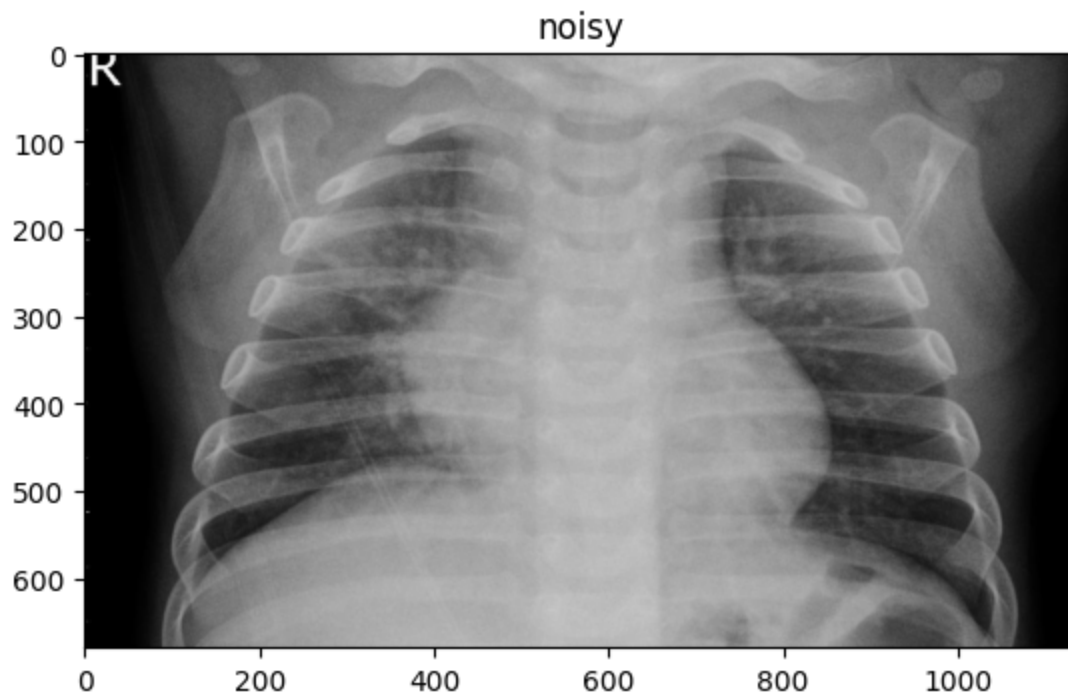
```

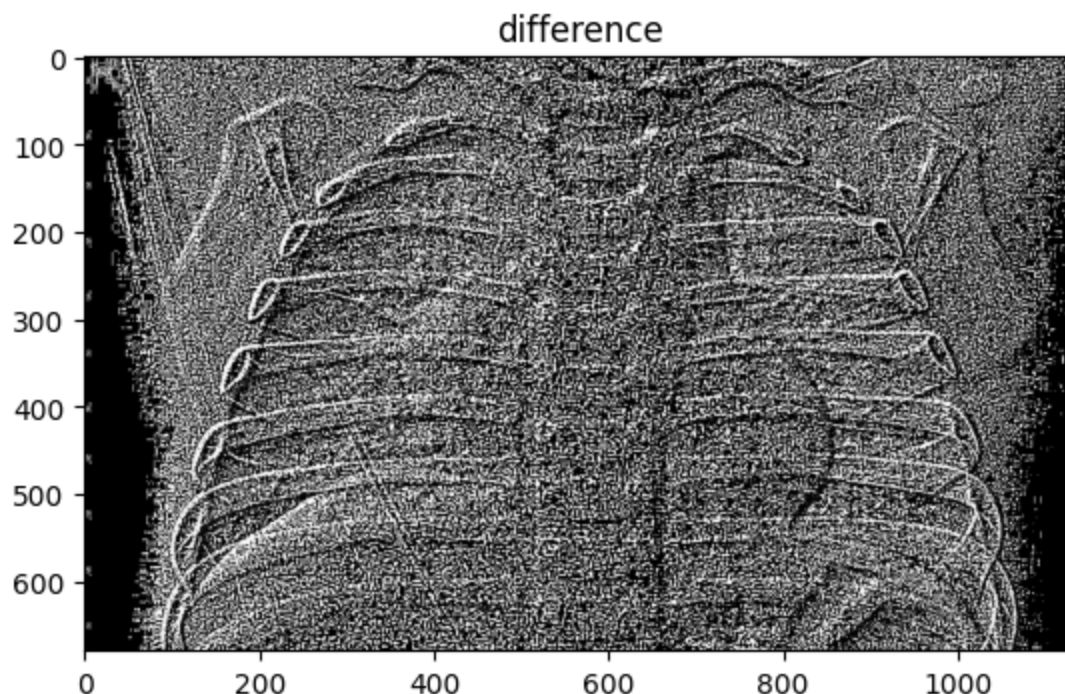
In [11]: noisy_img = Image.open("final_train/viral_pneumonia/viral_pneumonia_100.jpg")
plt.title("noisy")
plt.imshow(noisy_img, cmap="gray")
plt.show()

filtered_img = scipy.ndimage.median_filter(noisy_img, size=6)
plt.title("denoised")
plt.imshow(filtered_img, cmap="gray")
plt.show()

diff_img = np.abs(filtered_img - noisy_img)
plt.title("difference")
plt.imshow(diff_img, cmap="gray")
plt.show()

```





Data Preprocessing

This phase includes transforming the dataset into an acceptable format for the pretrained DEiT Model. Once the dataset has been modified, data augmentation is applied to the training set to improve generalizations on the validation set and test set

1. visualize image sizes of the entire dataset. This visualization reveals that the image sizes vary dramatically in all classes. This informs the resizing transformations seen in later steps.

```
In [13]: classes = ['viral_pneumonia', 'bacterial_pneumonia', 'no_pneumonia']
         ROOT_DIRECTORY = 'final_train/'

         vp_w, vp_h = [], []
         bp_w, bp_h = [], []
         np_w, np_h = [], []

         for label in classes:
             for filename in os.listdir(ROOT_DIRECTORY + label):
                 image = Image.open(ROOT_DIRECTORY + label + '/' + filename)
                 w, h = image.size

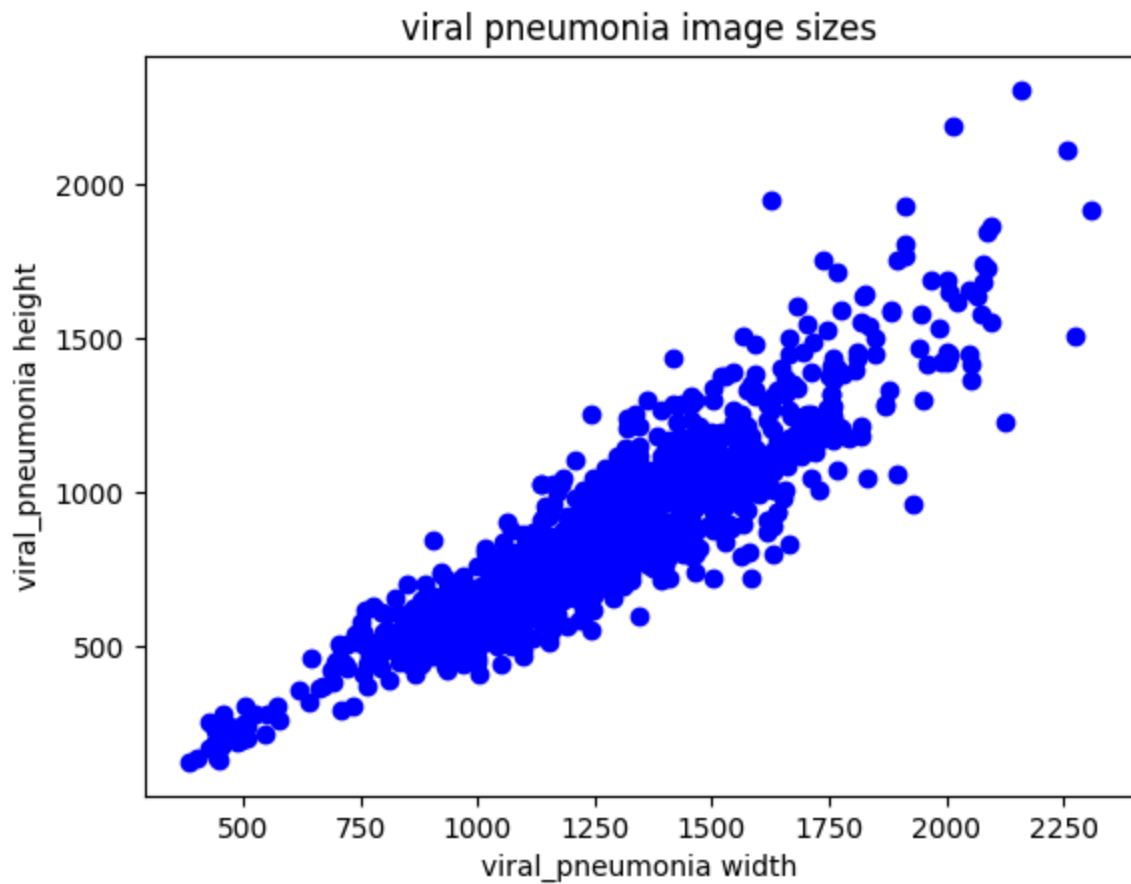
                 if(label == 'viral_pneumonia'):
                     vp_w.append(w)
                     vp_h.append(h)
                 elif(label == 'bacterial_pneumonia'):
                     bp_w.append(w)
                     bp_h.append(h)
                 else:
```

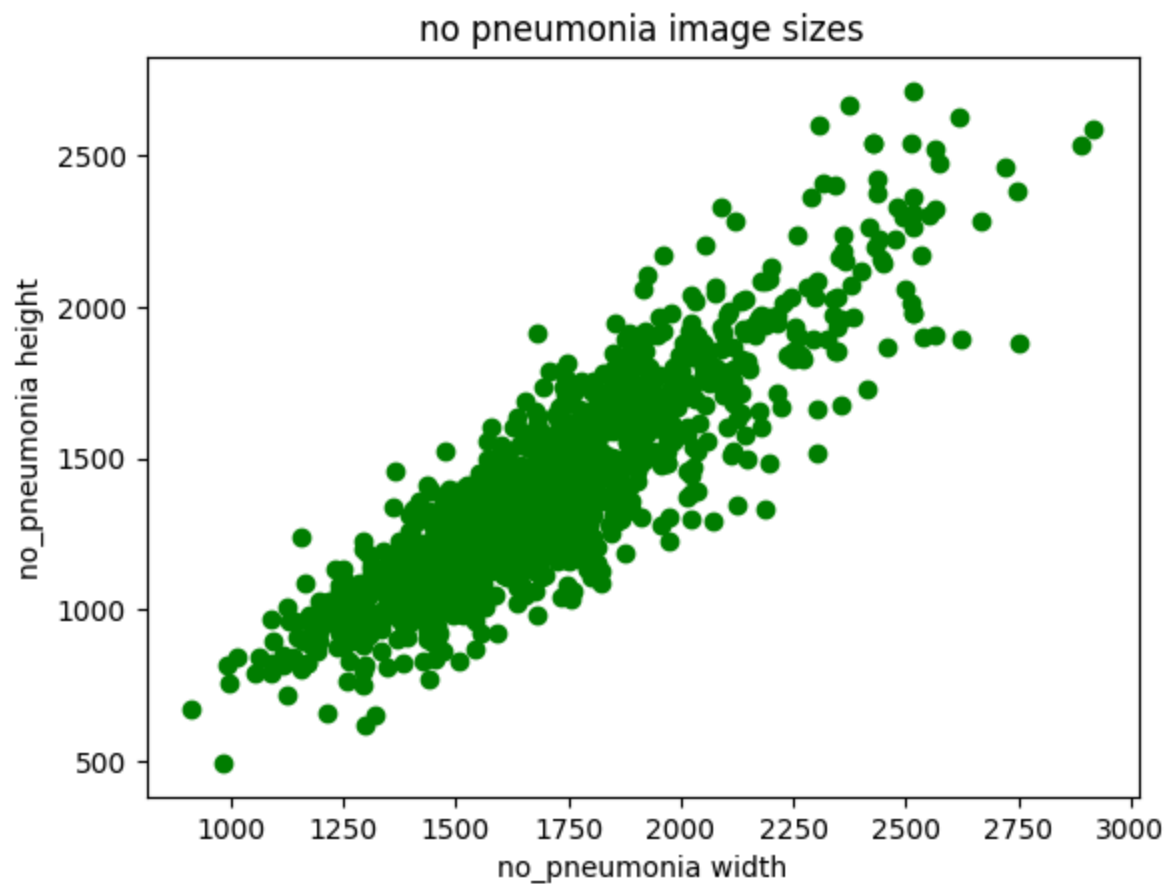
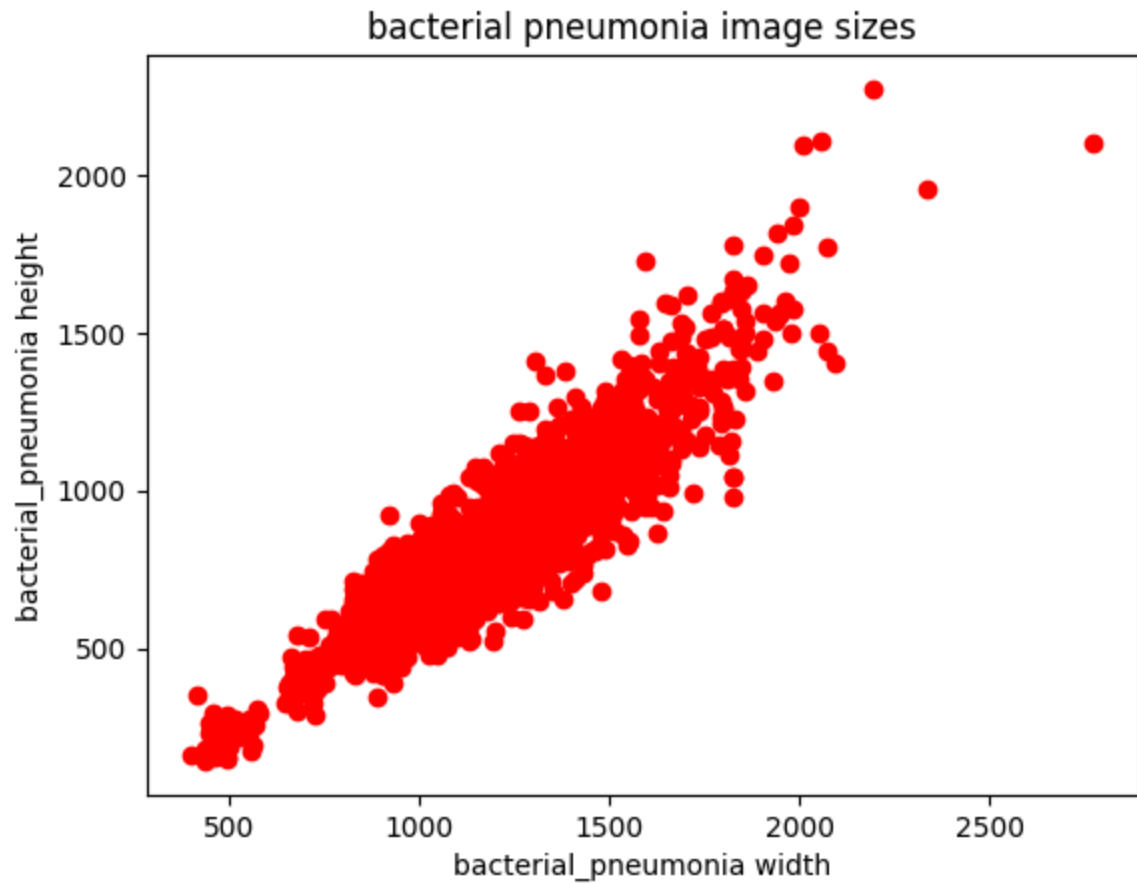
```
np_w.append(w)
np_h.append(h)

plt.scatter(vp_w, vp_h, color='blue')
plt.xlabel('viral_pneumonia width')
plt.ylabel('viral_pneumonia height')
plt.title("viral pneumonia image sizes")
plt.show()

plt.scatter(bp_w, bp_h, color='red')
plt.xlabel('bacterial_pneumonia width')
plt.ylabel('bacterial_pneumonia height')
plt.title("bacterial pneumonia image sizes")
plt.show()

plt.scatter(np_w, np_h, color='green')
plt.xlabel('no_pneumonia width')
plt.ylabel('no_pneumonia height')
plt.title("no pneumonia image sizes")
plt.show()
```





2. this class properly prepares the existing data into a format acceptable for the DEIT model. The class creates a dataset where each item is a pair of the image and its corresponding label.

```
In [30]: from torch.utils.data import Dataset

class createDataset(Dataset):
    def __init__(self, csv_file, img_dir, transform):
        self.data = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.transform = transform

        files = set(os.listdir(img_dir))

        self.data = self.data[self.data['file_name'].isin(files)].reset_index(drop=

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):
        img_name = self.data.iloc[index]['file_name']
        label = self.data.iloc[index]['class_id']

        img_path = os.path.join(self.img_dir, img_name)
        image = Image.open(img_path)

        if self.transform:
            image = self.transform(image)

        return image, label
```

3. Most preprocessing occurs here. Defining the data augmentation techniques for the training dataset, as well as normalization for the validation dataset. Using these transformations and dataset class above generates the appropriate dataloaders.

```
In [ ]: transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomRotation(45),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

transform_val = transforms.Compose([
    transforms.Resize(256),           # resize the shorter side to 256
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])
```

```

# === Load Dataset ===
full_dataset = ImageFolder(root='final_train', transform=transform)

# === Split Dataset ===
total_size = len(full_dataset)
train_size = int(0.8 * total_size)
val_size = int(0.1 * total_size)
test_size = total_size - train_size - val_size
generator = torch.Generator().manual_seed(42)

train_indices, val_indices, test_indices = random_split(
    range(total_size), [train_size, val_size, test_size]
)

# Use Subset with specific transforms for each split
train_dataset = torch.utils.data.Subset(ImageFolder(root='final_train', transform=tr
val_dataset = torch.utils.data.Subset(ImageFolder(root='final_train', transform=tra
test_dataset = torch.utils.data.Subset(ImageFolder(root='final_train', transform=tr

# === DataLoaders ===
train_dl = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=12)
val_dl = DataLoader(val_dataset, batch_size=128, shuffle=False, num_workers=12)
test_dl = DataLoader(test_dataset, batch_size=128, shuffle=False)

# === Class Balance Check ===
def count_classes(subset, name):
    labels = [subset.dataset.samples[i][1] for i in subset.indices]
    counts = Counter(labels)
    print(f"\n{name} class distribution:")
    for class_idx, class_name in enumerate(subset.dataset.classes):
        print(f"Class {class_idx} ({class_name}): {counts[class_idx]}")

count_classes(train_dataset, "Train")
count_classes(val_dataset, "Validation")
count_classes(test_dataset, "Test")

```

Train class distribution:
 Class 0 (bacterial_pneumonia): 1766
 Class 1 (no_pneumonia): 1001
 Class 2 (viral_pneumonia): 970

Validation class distribution:
 Class 0 (bacterial_pneumonia): 252
 Class 1 (no_pneumonia): 106
 Class 2 (viral_pneumonia): 109

Test class distribution:
 Class 0 (bacterial_pneumonia): 220
 Class 1 (no_pneumonia): 120
 Class 2 (viral_pneumonia): 128

Defining the Model

The solution model is the `deit_small_patch16_224` that expects inputs of size 224x224. The model breaks the image into 16 patches for embedding development. This transformer model is developed for smaller datasets. Because this model is a ViT, the following choices were made for (hyper)parameters:

- loss: crossentropyloss
- optimizer: adamW
- learning rate Scheduler: CosineLRScheduler

```
In [180... # consider a different model
import timm.loss
from timm.scheduler import CosineLRScheduler
from timm.data.mixup import Mixup

model = timm.create_model('deit_base_distilled_patch16_224', pretrained=True, num_c
model.head = nn.Linear(model.head.in_features, 3)

weights_tensor = weights_tensor.to(device)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1, weight=weights_tensor) # weigh
optimizer = opt.AdamW(model.parameters(), lr=5e-5, weight_decay=0.07) #
# scheduler = lr_scheduler.StepLR(optimizer, step_size=4, gamma=0.6)

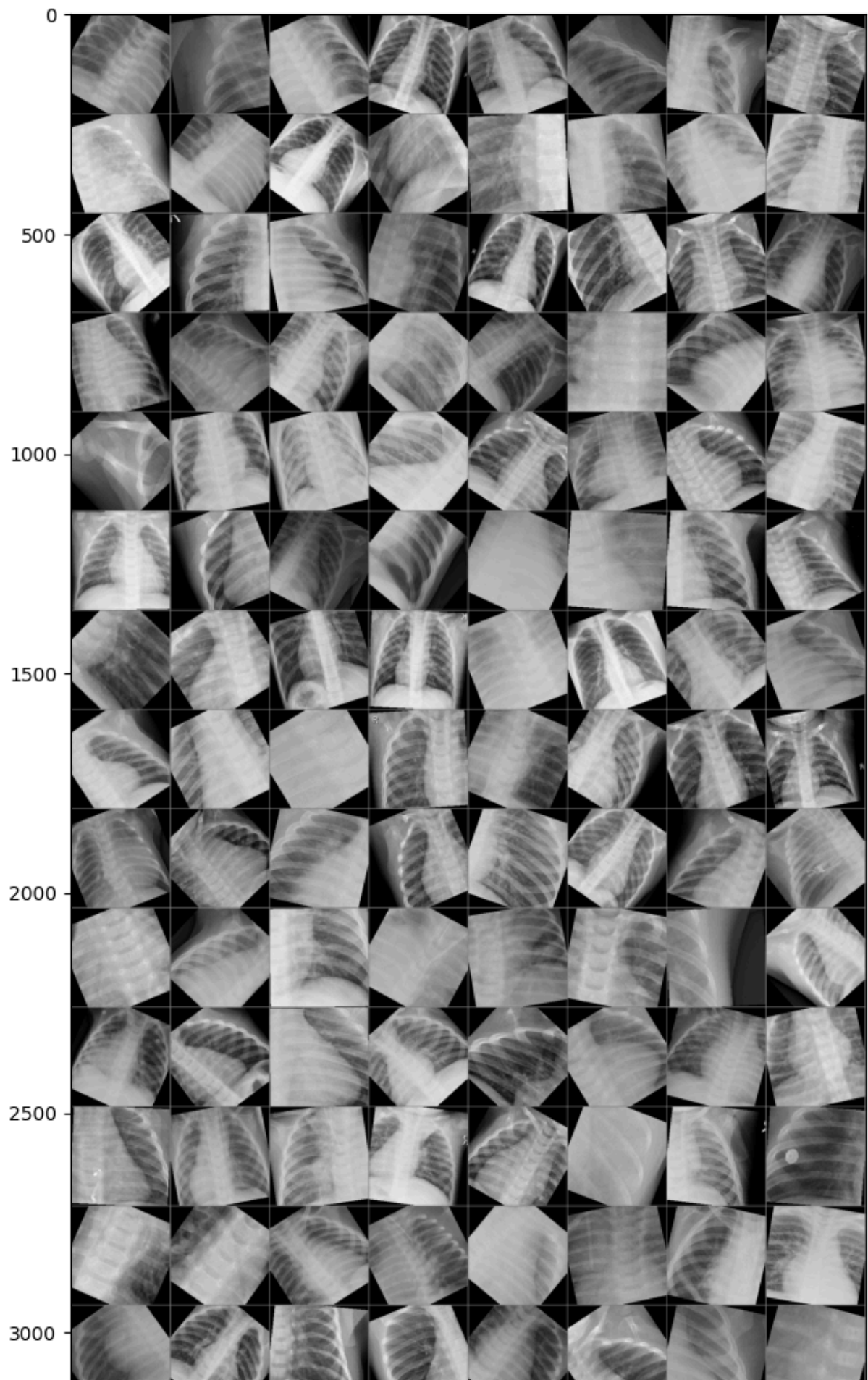
scheduler = CosineLRScheduler(
    optimizer,
    t_initial=50,          # total warmup+decay length
    lr_min=1e-6,
    warmup_lr_init=1e-6,   # start LR
    warmup_t=12,           # epochs of linear warmup
    cycle_limit=1,         # no restarts
)
```

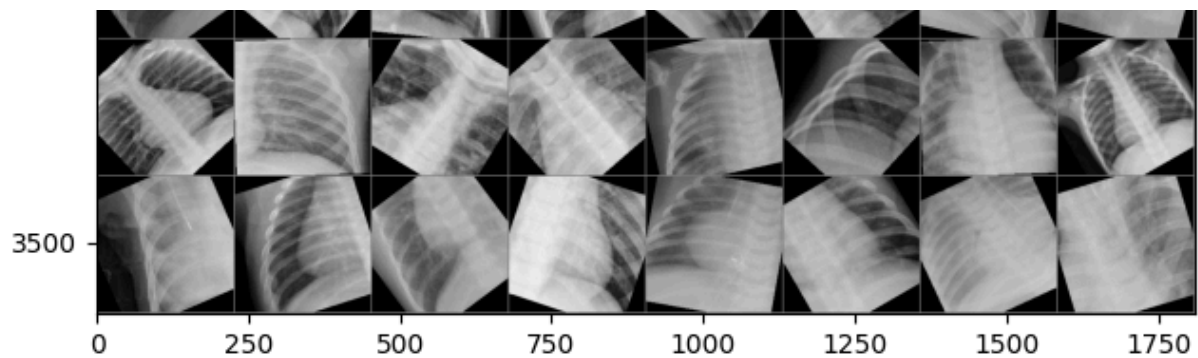
visualize the effects that the proposed data augmentation has on the dataset, and the images that the model will be training on.

```
In [181... fig = plt.figure(figsize=(10, 15))
def imshow(img):
    img = img / 2 + 0.5     # denormalize change this
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))

# get some random training images
dataiter = iter(train_dl)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
```





Training & Evaluation Loop

The training loop starts by defining an accuracy function and evaluation function to be used during the training process. This keeps the model modular. The following training loop occurs for 50 epochs and follows typical training loop conventions for most classification models.

```
In [182... from sklearn.metrics import accuracy_score

def get_accuracy(predicted, labels):
    batch_len, correct = 0, 0
    batch_len = labels.size(0)
    correct = (predicted == labels).sum().item()
    return batch_len, correct

def evaluate(model, val_loader):
    losses = 0
    num_samples_total = 0
    correct_total = 0
    model.eval()
    for inputs, labels in val_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        out = model(inputs)
        _, predicted = torch.max(out, 1)
        loss = criterion(out, labels)
        losses += loss.item()
        b_len, corr = get_accuracy(predicted, labels)
        num_samples_total += b_len
        correct_total += corr
    accuracy = correct_total / num_samples_total
    losses = losses / len(val_loader)
    return losses, accuracy
```

```
In [183... # training loop
num_epochs = 50
accuracy = []
val_accuracy = []
losses = []
val_losses = []
```



```

device = torch.device('cuda:3' if torch.cuda.is_available() else 'cpu')
weight_tensors = torch.tensor(
    [class_weights[c] for c in sorted_classes],
    dtype=torch.float
).to(device)
model = model.to(device)

print(f"using {device}")
for epoch in range(num_epochs):
    running_loss = 0.0
    correct_total = 0.0
    num_samples_total = 0.0
    for i, data in enumerate(train_dl):
        # get the inputs
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        # set the parameter gradients to zero
        optimizer.zero_grad()

        # forward + backward + optimize
        # logits, logits_dist = model(inputs) # distill exclusive
        # avg_logits = (logits + logits_dist) * 0.5 # distill exclusive
        # loss = criterion(avg_logits, labels) # distill exclusive

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        #compute accuracy
        _, predicted = torch.max(outputs, 1) # change to outputs for non distilled
        b_len, corr = get_accuracy(predicted, labels)
        num_samples_total += b_len
        correct_total += corr
        running_loss += loss.item()

    scheduler.step(epoch)
    running_loss /= len(train_dl)
    train_accuracy = correct_total / num_samples_total
    val_loss, val_acc = evaluate(model, val_dl)

    print('Epoch: %d' % (epoch+1))
    print('Loss: %.3f Accuracy: %.3f' % (running_loss, train_accuracy))
    print('Validation Loss: %.3f Val Accuracy: %.3f' % (val_loss, val_acc))

    losses.append(running_loss)
    val_losses.append(val_loss)
    accuracy.append(train_accuracy)
    val_accuracy.append(val_acc)
print('Finished Training')

```

using cuda:3

Epoch: 1
Loss: 1.037 Accuracy:0.468
Validation Loss: 0.948 Val Accuracy: 0.542
Epoch: 2
Loss: 0.979 Accuracy:0.478
Validation Loss: 0.915 Val Accuracy: 0.559
Epoch: 3
Loss: 0.896 Accuracy:0.547
Validation Loss: 0.792 Val Accuracy: 0.741
Epoch: 4
Loss: 0.766 Accuracy:0.687
Validation Loss: 0.699 Val Accuracy: 0.756
Epoch: 5
Loss: 0.737 Accuracy:0.727
Validation Loss: 0.669 Val Accuracy: 0.771
Epoch: 6
Loss: 0.700 Accuracy:0.759
Validation Loss: 0.795 Val Accuracy: 0.719
Epoch: 7
Loss: 0.709 Accuracy:0.746
Validation Loss: 0.691 Val Accuracy: 0.762
Epoch: 8
Loss: 0.682 Accuracy:0.766
Validation Loss: 0.639 Val Accuracy: 0.792
Epoch: 9
Loss: 0.673 Accuracy:0.779
Validation Loss: 0.618 Val Accuracy: 0.820
Epoch: 10
Loss: 0.672 Accuracy:0.779
Validation Loss: 0.656 Val Accuracy: 0.779
Epoch: 11
Loss: 0.653 Accuracy:0.783
Validation Loss: 0.614 Val Accuracy: 0.816
Epoch: 12
Loss: 0.649 Accuracy:0.785
Validation Loss: 0.654 Val Accuracy: 0.788
Epoch: 13
Loss: 0.658 Accuracy:0.782
Validation Loss: 0.641 Val Accuracy: 0.799
Epoch: 14
Loss: 0.633 Accuracy:0.800
Validation Loss: 0.604 Val Accuracy: 0.820
Epoch: 15
Loss: 0.632 Accuracy:0.798
Validation Loss: 0.644 Val Accuracy: 0.792
Epoch: 16
Loss: 0.615 Accuracy:0.809
Validation Loss: 0.632 Val Accuracy: 0.797
Epoch: 17
Loss: 0.632 Accuracy:0.795
Validation Loss: 0.603 Val Accuracy: 0.820
Epoch: 18
Loss: 0.614 Accuracy:0.812
Validation Loss: 0.626 Val Accuracy: 0.807
Epoch: 19
Loss: 0.624 Accuracy:0.808

Validation Loss: 0.606 Val Accuracy: 0.812
Epoch: 20
Loss: 0.606 Accuracy:0.813
Validation Loss: 0.584 Val Accuracy: 0.829
Epoch: 21
Loss: 0.592 Accuracy:0.823
Validation Loss: 0.600 Val Accuracy: 0.814
Epoch: 22
Loss: 0.590 Accuracy:0.825
Validation Loss: 0.598 Val Accuracy: 0.816
Epoch: 23
Loss: 0.600 Accuracy:0.822
Validation Loss: 0.581 Val Accuracy: 0.835
Epoch: 24
Loss: 0.581 Accuracy:0.821
Validation Loss: 0.575 Val Accuracy: 0.824
Epoch: 25
Loss: 0.579 Accuracy:0.832
Validation Loss: 0.580 Val Accuracy: 0.827
Epoch: 26
Loss: 0.578 Accuracy:0.832
Validation Loss: 0.556 Val Accuracy: 0.839
Epoch: 27
Loss: 0.561 Accuracy:0.841
Validation Loss: 0.571 Val Accuracy: 0.833
Epoch: 28
Loss: 0.555 Accuracy:0.843
Validation Loss: 0.583 Val Accuracy: 0.837
Epoch: 29
Loss: 0.554 Accuracy:0.845
Validation Loss: 0.577 Val Accuracy: 0.816
Epoch: 30
Loss: 0.542 Accuracy:0.850
Validation Loss: 0.611 Val Accuracy: 0.829
Epoch: 31
Loss: 0.547 Accuracy:0.859
Validation Loss: 0.597 Val Accuracy: 0.829
Epoch: 32
Loss: 0.543 Accuracy:0.852
Validation Loss: 0.580 Val Accuracy: 0.837
Epoch: 33
Loss: 0.534 Accuracy:0.864
Validation Loss: 0.582 Val Accuracy: 0.831
Epoch: 34
Loss: 0.528 Accuracy:0.863
Validation Loss: 0.615 Val Accuracy: 0.809
Epoch: 35
Loss: 0.530 Accuracy:0.854
Validation Loss: 0.586 Val Accuracy: 0.833
Epoch: 36
Loss: 0.520 Accuracy:0.871
Validation Loss: 0.583 Val Accuracy: 0.829
Epoch: 37
Loss: 0.520 Accuracy:0.868
Validation Loss: 0.618 Val Accuracy: 0.822
Epoch: 38

```

Loss: 0.524 Accuracy:0.864
Validation Loss: 0.586 Val Accuracy: 0.829
Epoch: 39
Loss: 0.506 Accuracy:0.877
Validation Loss: 0.594 Val Accuracy: 0.829
Epoch: 40
Loss: 0.502 Accuracy:0.877
Validation Loss: 0.587 Val Accuracy: 0.818
Epoch: 41
Loss: 0.495 Accuracy:0.882
Validation Loss: 0.581 Val Accuracy: 0.816
Epoch: 42
Loss: 0.511 Accuracy:0.876
Validation Loss: 0.592 Val Accuracy: 0.827
Epoch: 43
Loss: 0.507 Accuracy:0.878
Validation Loss: 0.588 Val Accuracy: 0.824
Epoch: 44
Loss: 0.501 Accuracy:0.873
Validation Loss: 0.588 Val Accuracy: 0.831
Epoch: 45
Loss: 0.500 Accuracy:0.883
Validation Loss: 0.588 Val Accuracy: 0.829
Epoch: 46
Loss: 0.498 Accuracy:0.880
Validation Loss: 0.586 Val Accuracy: 0.824
Epoch: 47
Loss: 0.492 Accuracy:0.885
Validation Loss: 0.587 Val Accuracy: 0.824
Epoch: 48
Loss: 0.487 Accuracy:0.886
Validation Loss: 0.585 Val Accuracy: 0.824
Epoch: 49
Loss: 0.487 Accuracy:0.888
Validation Loss: 0.585 Val Accuracy: 0.824
Epoch: 50
Loss: 0.499 Accuracy:0.889
Validation Loss: 0.586 Val Accuracy: 0.831
Finished Training

```

training visualization and loss curves

Model Analysis

1. Visualize the training/validation results. The graphs showcase a desirable outcome for both training and validation accuracy and loss. Both trend positively together, indicating a low chance of overfitting in the model.

In [184...

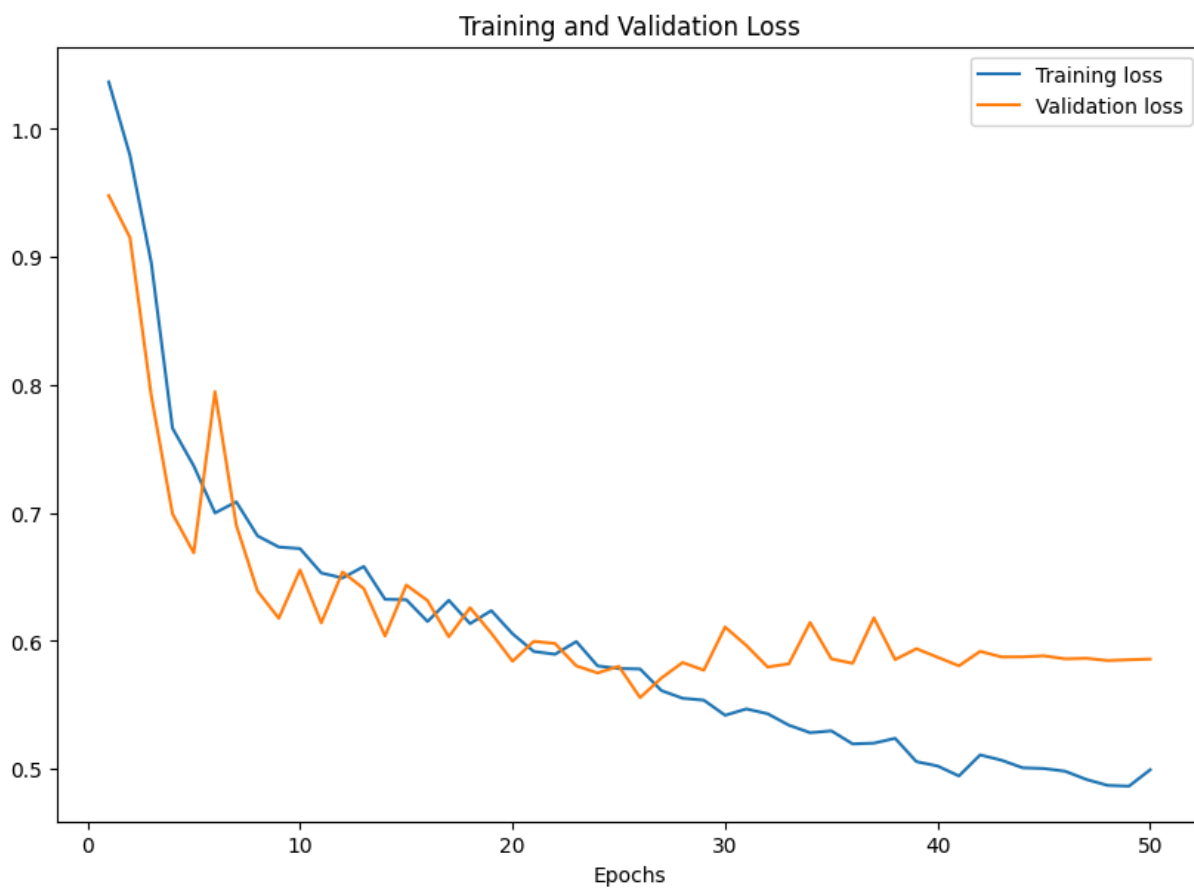
```

epoch = range(1, num_epochs+1)
fig = plt.figure(figsize=(10, 15))
plt.subplot(2,1,2)
plt.plot(epoch, losses, label='Training loss')
plt.plot(epoch, val_losses, label='Validation loss')

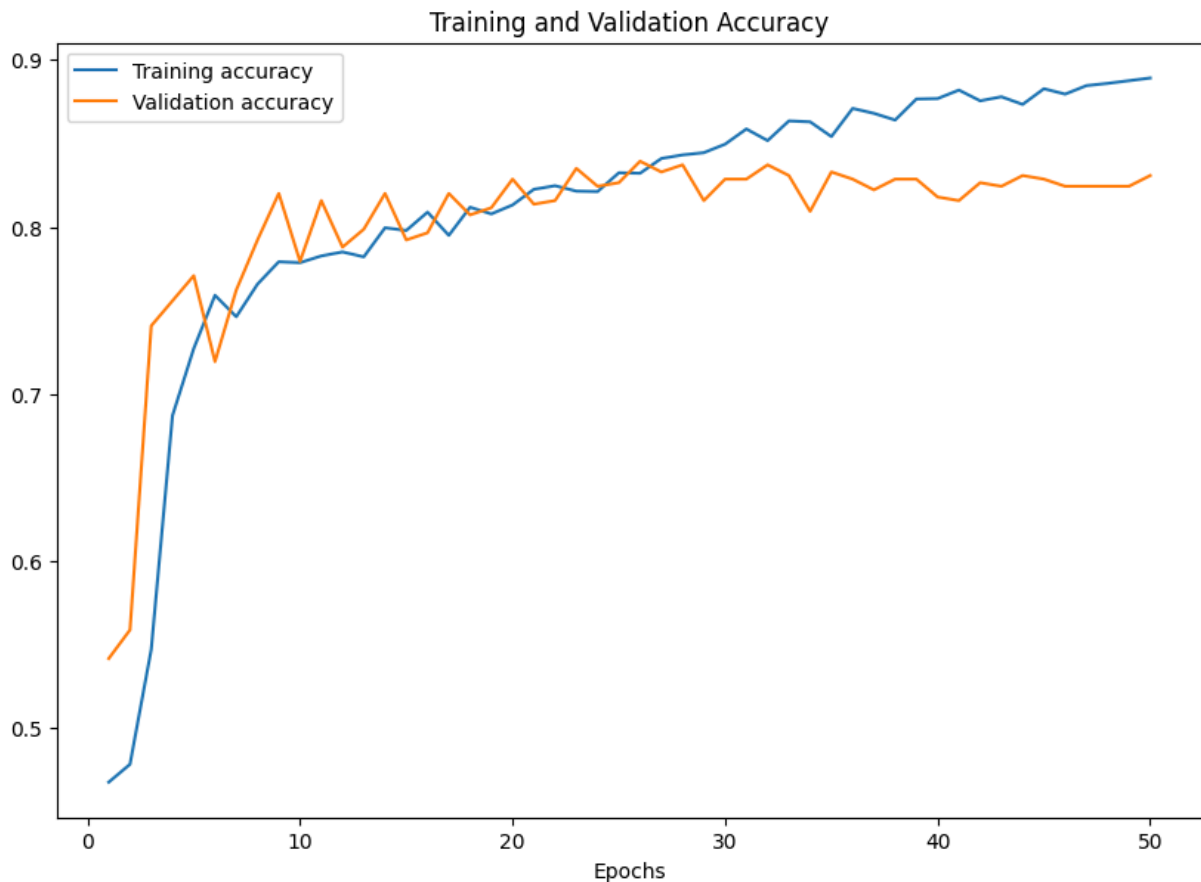
```

```
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.legend()
plt.figure()
plt.show()

fig = plt.figure(figsize=(10, 15))
plt.subplot(2,1,2)
plt.plot(epoch, accuracy, label='Training accuracy')
plt.plot(epoch, val_accuracy, label='Validation accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.legend()
plt.figure()
plt.show()
```



<Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

- the confusion matrix demonstrates the alignment of predicted and true labels. The solution matrix demonstrates a well defined diagonal line. This diagonal indicates that the predicted labels and true labels match well. Note that the model did have problems with the viral samples seen in the bottom right of the matrix. As mentioned before in the data visualization section, this occurs because viral samples, especially in earlier phases, are similar to normal samples ultimately reducing the models accuracy for viral samples.

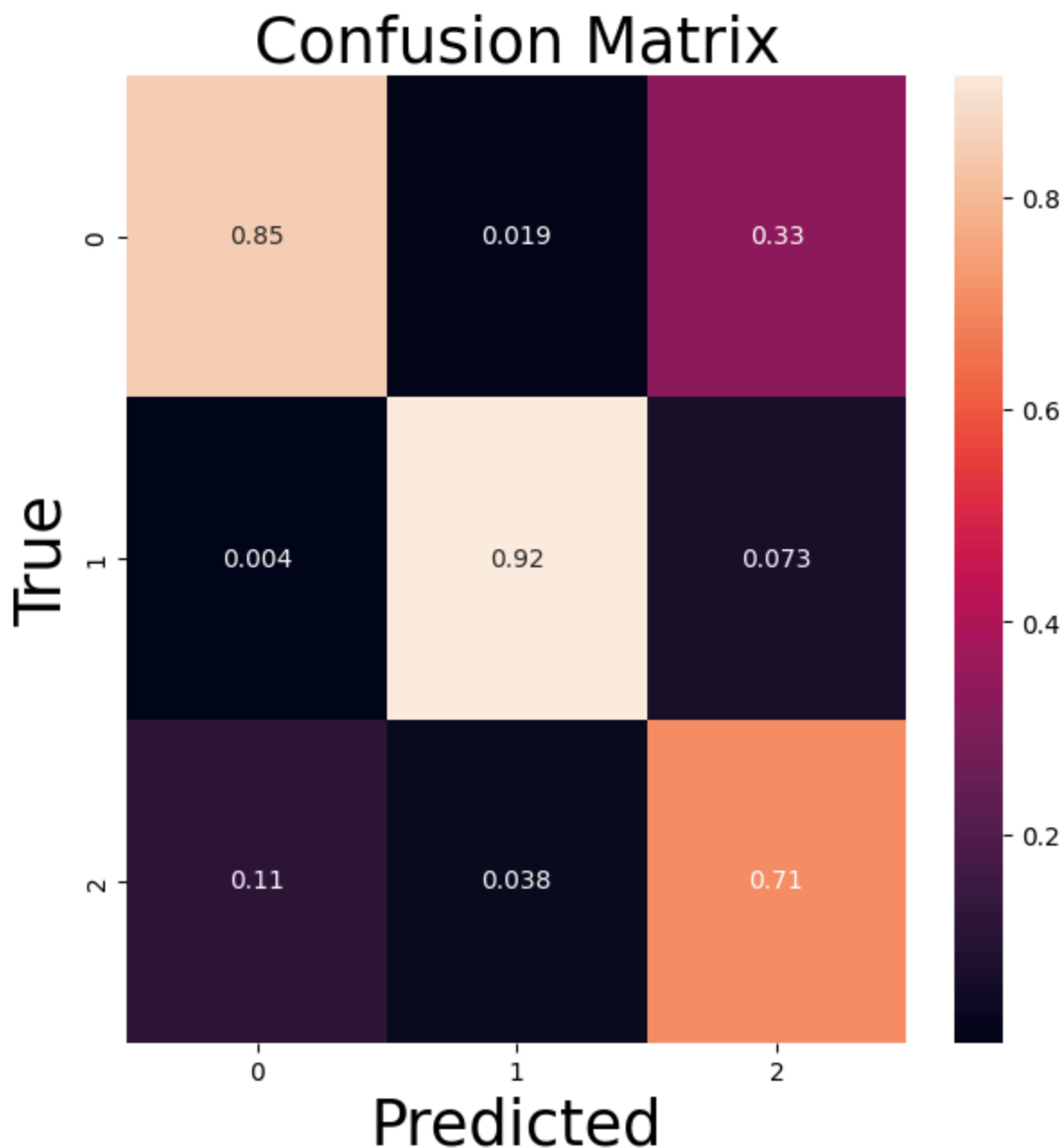
```
In [185... confusion_matrix = torch.zeros(len(classes), len(classes))
model = model.to(device)
with torch.no_grad():
    for data in val_dl:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        for t, p in zip(labels.view(-1), predicted.view(-1)):
            confusion_matrix[t.long(), p.long()] += 1

cm = confusion_matrix.numpy()

fig, ax = plt.subplots(figsize=(7, 7))
sns.heatmap(cm / (cm.astype(float).sum(axis=1) + 1e-9), annot=True, ax=ax)
```

```
# Labels, title and ticks
ax.set_xlabel('Predicted', size=25)
ax.set_ylabel('True', size=25)
ax.set_title('Confusion Matrix', size=25)
```

Out[185... Text(0.5, 1.0, 'Confusion Matrix')



```
In [ ]: confusion_matrix = torch.zeros(len(classes), len(classes))
model = model.to(device)
with torch.no_grad():
    for data in test_dl:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        for t, p in zip(labels.view(-1), predicted.view(-1)):
            confusion_matrix[t.long(), p.long()] += 1
```

```

cm = confusion_matrix.numpy()

fig,ax= plt.subplots(figsize=(7,7))
sns.heatmap(cm / (cm.astype(float).sum(axis=1) + 1e-9), annot=True, ax=ax)

# Labels, title and ticks
ax.set_xlabel('Predicted', size=25)
ax.set_ylabel('True', size=25)
ax.set_title('Confusion Matrix', size=25)

```

3. The following code takes the newly trained model and performs inference on the test dataset. This step is run last and after finetuning as to not expose the test dataset to the training process.

```

In [ ]: #variables for counting correct predictions
correct = 0

#total predictions
total = 0
model.eval()
with torch.no_grad():
    #Loop for data in the test dataset
    for data in test_dl:
        images, labels = data
        #move input data and labels to device (cuda/GPU or CPU)
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        #compute accuracy
        _, predicted = torch.max(outputs.data, 1)
        #get total number of labels
        total += labels.size(0)
        #add up correct predictions
        correct += (predicted == labels).sum().item()

#print accuracy
print('Accuracy of the network on the val images: %d %%' % (
    100 * correct / total))

```

4. The following gradCAM implementation was largely adapted from the SkinLesion class project in COMP.5300 Computing for Health & Medicine. The GradCAM Informs model reliability as it highlights the main areas influencing the prediction of certain labels. If these heatmaps appear in expected locations consistently, the model proves to be reliable.

```

In [187... from collections import OrderedDict

class _BaseWrapper(object):
    """
    Please modify forward() and backward() according to your task.
    """

```



```

def __init__(self, model):
    super(_BaseWrapper, self).__init__()
    self.device = next(model.parameters()).device
    self.model = model
    self.handlers = [] # a set of hook function handlers

def _encode_one_hot(self, ids):
    one_hot = torch.zeros_like(self.logits).to(self.device)
    one_hot.scatter_(1, ids, 1.0)
    return one_hot

def forward(self, image):
    """
    Simple classification
    """
    self.model.zero_grad()
    self.logits = self.model(image)
    self.probs = F.softmax(self.logits, dim=1)
    return self.probs.sort(dim=1, descending=True)

def backward(self, ids):
    """
    Class-specific backpropagation
    Either way works:
    1. self.logits.backward.gradient=one_hot, retain_graph=True)
    2. (self.logits * one_hot).sum().backward(retain_graph=True)
    """

    one_hot = self._encode_one_hot(ids)
    self.logits.backward(gradient=one_hot, retain_graph=True)

def generate(self):
    raise NotImplementedError

def remove_hook(self):
    """
    Remove all the forward/backward hook functions
    """
    for handle in self.handlers:
        handle.remove()

class GradCAM(_BaseWrapper):
    """
    "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localizati
    https://arxiv.org/pdf/1610.02391.pdf
    Look at Figure 2 on page 4
    """

    def __init__(self, model, candidate_layers=None):
        super(GradCAM, self).__init__(model)
        self.fmap_pool = OrderedDict()
        self.grad_pool = OrderedDict()
        self.candidate_layers = candidate_layers # list

```

```

def forward_hook(key):
    def forward_hook_(module, input, output):
        # Save featuremaps
        self.fmap_pool[key] = output.detach()

    return forward_hook_

def backward_hook(key):
    def backward_hook_(module, grad_in, grad_out):
        # Save the gradients correspond to the featuremaps
        self.grad_pool[key] = grad_out[0].detach()

    return backward_hook_

# If any candidates are not specified, the hook is registered to all the la
for name, module in self.model.named_modules():
    if self.candidate_layers is None or name in self.candidate_layers:
        self.handlers.append(module.register_forward_hook(forward_hook(name
        self.handlers.append(module.register_backward_hook(backward_hook(na

def _find(self, pool, target_layer):
    if target_layer in pool.keys():
        return pool[target_layer]
    else:
        raise ValueError("Invalid layer name: {}".format(target_layer))

def _compute_grad_weights(self, grads):
    return F.adaptive_avg_pool2d(grads, 1)

def forward(self, image):
    self.image_shape = image.shape[2:]
    return super(GradCAM, self).forward(image)

def generate(self, target_layer):
    fmaps = self._find(self.fmap_pool, target_layer)
    grads = self._find(self.grad_pool, target_layer)
    weights = self._compute_grad_weights(grads)

    gcam = torch.mul(fmaps, weights).sum(dim=1, keepdim=True)
    gcam = F.relu(gcam)

    gcam = F.interpolate(
        gcam, self.image_shape, mode="bilinear", align_corners=False
    )

    B, C, H, W = gcam.shape
    gcam = gcam.view(B, -1)
    gcam -= gcam.min(dim=1, keepdim=True)[0]
    gcam /= gcam.max(dim=1, keepdim=True)[0]
    gcam = gcam.view(B, C, H, W)

    return gcam

```

```

In [ ]: import torch
import numpy as np
import matplotlib.pyplot as plt

```

```

import cv2

norm_mean = [0.485, 0.456, 0.406]
norm_std = [0.229, 0.224, 0.225]

def demo2(image, label, model):
    """
    Generate Grad-CAM with overlay
    """
    model.to(device)
    model.eval()

    target_layers = ["patch_embed.proj"]
    target_class = label.item()

    images = image.unsqueeze(0)
    gcam = GradCAM(model=model)
    probs, ids = gcam.forward(images)
    ids_ = torch.LongTensor([[target_class]] * len(images)).to(device)
    gcam.backward(ids=ids_)

    for target_layer in target_layers:
        print("Generating Grad-CAM @{}".format(target_layer))
        regions = gcam.generate(target_layer=target_layer)

    for j in range(len(images)):
        print(
            "\t#{:}: {} ({:.5f})".format(
                j, classes[target_class], float(probs[0, target_class])
            )
        )

        # Grad-CAM heatmap
        gcam_region = regions[j, 0].cpu().numpy()
        gcam_resized = cv2.resize(gcam_region, (image.shape[2], image.shape[1]))
        heatmap = cv2.applyColorMap(np.uint8(255 * gcam_resized), cv2.COLORMAP_
        heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)

        # Unnormalize image
        image_np = image.cpu().numpy().transpose(1, 2, 0)
        image_unnorm = (image_np * np.array(norm_std)) + np.array(norm_mean)
        image_unnorm = np.clip(image_unnorm, 0, 1)

        # Convert to uint8
        image_uint8 = np.uint8(255 * image_unnorm)

        # Overlay
        overlay = cv2.addWeighted(image_uint8, 0.7, heatmap, 0.3, 0)

        # Show all
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 3, 1)
        plt.imshow(image_unnorm)
        plt.title("Unnormalized Image")
        plt.axis("off")

```

```

plt.subplot(1, 3, 2)
plt.imshow(gcam_resized, cmap='jet')
plt.title("Grad-CAM Heatmap")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(overlay)
plt.title("Overlay")
plt.axis("off")

plt.suptitle(f"Predicted: {classes[target_class]}", fontsize=14)
plt.tight_layout()
plt.show()

# --- Run Grad-CAM for one image per class ---

# Track which classes we've seen
seen_classes = {cls: False for cls in range(len(classes))}

# Load the model
model = model

# Loop through validation data
for images, labels in test_dl:
    for i in range(len(labels)):
        label = labels[i].item()
        if not seen_classes[label]:
            print(f"\nTrue Class: {classes[label]}")
            image = images[i].to(device)
            label_tensor = labels[i].to(device)

            # Generate Grad-CAM visualization
            demo2(image, label_tensor, model)

            seen_classes[label] = True

# Exit loop when we've visualized all classes
if all(seen_classes.values()):
    break

image, label = next(iter(test_dl))
# Load the model
model = model
# Grad cam
demo2(image[0].to(device), label[0].to(device), model)

image = np.transpose(image[0], (1,2,0))
image2 = np.add(np.multiply(image.numpy(), np.array(norm_std)), np.array(norm_mean))
print("True Class: ", classes[label[0].cpu()])
plt.imshow(image)
plt.show()

```

```
plt.imshow(image2)  
plt.show()
```